



Scripting System for Ember LIMS

Design and Implementation of a secure C# Scripting Layer in a .NET ASP Application

Bachelor Project Report

Bachelor in Applied Information Technology

Professional Internship at Labsolution S.à.r.l.

Student name

Conrad Gilles (023198615A)

Names of the instructors

(Ohlhoff Claude) – professional supervisor

(Grevisse Christian) – academic supervisor

Date of submission

01.06.2026

Abstract.....	2
Introduction.....	3
Project Context and requirements	4
The Legacy LX System and Python Limitations	4
The Ember Migration and .NET 9.....	4
Proof of Concept Objectives	5
Architecture and System Design.....	7
Separation of the Scripting Framework and API.....	7
Architectural Principles	9
Design Patterns Used:	11
System Components.....	13
Implementation	16
Script Compilation and Validation.....	16
Script Execution	21
ScriptRepository and ScriptManagerBase	23
The ScriptManager	23
The scripting API in detail.....	25
The Versioning System.....	27
Validation and Evaluation	37
Limitations and Future Work.....	40
Conclusion.....	43
References	44

Abstract

This bachelor thesis describes the design and implementation of a secure, ahead-of-time compiled C# scripting system for a new Laboratory Information Management System (LIMS) called Ember. The transition from the previous product's Python-based architecture to .NET 9 was necessary, as there was a need for strict compile-time validation, type safety, and improved technology stack alignment. The scripting system aims to enable technically proficient customers to customize the LIMS securely without having to modify the source code.

The system utilizes the Roslyn compiler platform to perform multiple validations on the source code of each script before scripts are inserted into the database, this helps prevent a multitude of runtime exceptions while also preventing unauthorized access to the Ember application. To address requirements such as backwards compatibility and multiple environments such as Test, Acceptance, and Production that share the same database, the architecture includes a modular API versioning mechanism. This is achieved by the implementation of a runtime context downgrade strategy combined with a custom return type upgrade strategy.

The resulting Proof-of-Concept that relies heavily on principles such as Clean Architecture and Domain-Driven Design, successfully demonstrates that dynamic C# script compilation and execution can be safely implemented within the Ember application domain. The final system showcases a performant, modular, and strongly typed foundation that solves the issues of the previous system and successfully helps Ember prepare for its production deployment in the future.

Introduction

My bachelor project was within the context of a major system migration at my host company where the previous and still active product called LX, which is a Laboratory Information Management System (LIMS) is being replaced by a more modern product called Ember. Because the previous system relied on Python for its backend scripting language, the lack of compile-time validation allowed customers to write and execute unsafe scripts. Therefore, to address the missing security and versioning features while also improving technology stack alignment, the company decided to create a new scripting system using the .Net framework.

During the 12-week internship, I designed and implemented a proof-of-concept of this new scripting system in C#. The main goal of the project and scripts in general is to allow technically proficient customers to customize the behavior of Ember without having to modify the underlying source code. Another requirement is that the system also needs to ensure that customer-written scripts are as safe and stable as possible. Therefore, the new system, unlike the previous Python implementation, needs to perform a variety of validations on the source code of a script before it is ever inserted into the database or executed. These validations are essential as they will prevent a large amount of runtime exceptions in the Ember product.

Furthermore, because Ember will run simultaneously in different environments on the client side (Test, Acceptance and Production) that access the same database, allowing for safe backwards compatibility was a crucial requirement. To solve this, the architecture of the system that I created is relying heavily on programming principles such as Clean Architecture and Domain-Driven Design and employs an ahead-of-time compilation strategy using Roslyn. The versioning mechanism is based on a context downgrade strategy that works by translating modern API calls so they match older scripts. This ensures that developers have strict control over how old scripts are executed.

This thesis focuses only on the architecture, implementation, and evaluation of the C# scripting system. By explaining in detail the difference between the Scripting Framework and the API, the compiler and validations, and the robust execution of the scripts, I will demonstrate how this proof of concept successfully provides a performant, reliable, and future-proof foundation for the future of the Ember product.

Project Context and requirements

The Legacy LX System and Python Limitations

My bachelor project was completed within the context of a major system migration at my host company. The current product, a so-called Laboratory Information Management System (LIMS) called LX, was built on a rather old and outdated technology called Uniface and is currently being replaced by a more modern product called Ember, which is built using .Net 9. Although the previous backend and its scripting system were written in Python, which of course, is still a highly relevant and modern language, the company decided to rewrite the scripting system to achieve better technology stack alignment, ensuring the entire codebase is written in C#.

The old Python system was simple and did not offer much complexity, which can be an advantage, but in the case of the LX product this lack of complexity led to a lot of desired features missing, notably security and versioning features. In addition to the simple design of the former system, Python's dynamic typing and lack of compile-time validation allowed users to easily write and execute unsafe scripts.

C#, which is the primary language for the new Ember project, offers not only strict type safety but also performance benefits and arguably better IDE support and debugging possibilities.

The Ember Migration and .NET 9

The new product, which includes my scripting system, is written using a much more modern framework that is safer, more timeproof, and doesn't require the use of multiple programming languages for the backend and the front-end. The framework chosen by my host company was the .Net framework, specifically .Net 9, which is quite modern since it was released in November 2024.

The .Net framework was not only chosen for its robust type safety using C# but also because it offers the use of C# combined with HTML to create a front-end application using Blazor. This is a huge advantage because it will allow the developers to keep using their existing C# skills to create the front-end without having to worry about working with multiple languages at the same time.

The system I created over the 12-week period is a proof of concept of a new scripting system that can be integrated into the new Ember product, it is designed to allow customers with technical knowledge to customize the behavior of Ember without having to modify the underlying source code or redeploy the application.

Proof of Concept Objectives

Security

The primary objective of the scripting proof of concept is to ensure that customer-written scripts are as safe as possible to prevent unwanted behavior and ensure a good customer experience. As the previous system was written in Python, there was basically zero validation being done before the insertion of a script by a customer. This, of course is very unsafe as it can lead to unchecked runtime exceptions being thrown. To avoid this, the new scripting system should only allow users to insert scripts into the database that are carefully checked for not only syntax errors but also for errors in their logic.

For example, the validation logic I wrote blocks any script that defines more than one class, as in the system I created, one class represents one script. Therefore, allowing multiple classes per database entry would violate this architecture and cause instability.

Versioning and Modularity

Another key requirement was ensuring backwards compatibility for when the scripting system is modified in the future. This is crucial because, as the scripting system might one day be deeply integrated into the future product, it needs to stay modular, as the requirements for Ember will undoubtedly change in the coming years.

Further complicating the support for backwards compatibility is that Ember will need to run in environments where multiple versions are deployed at the same time while also sharing the same database. This methodology is based on the previous product LX, where there are typically three different versions that the customer can run.

1. Test (T), serves to verify if the product works as expected
2. Acceptance (A), serves to allow the customer to test the product in his environment and verify if it meets his expectations
3. Production (P), the product is deployed and available to all users within the system

(Development, testing, acceptance and production, 2026) (Evans C. , 2009)

In the LX setup, Acceptance and Production point to the same database and therefore can also access the same data even though they might be using a different version of the scripting API.

This introduces another issue concerning the compatibility of updated or newly written scripts, for example, if a customer writes a script that is based on a new version of the scripting API that is

only available in the Test version, and this script is somehow executed in the Production version it will throw a runtime error.

Similarly, if a customer updates a script to the new scripting API that is available in the Production version, and the script is now somehow called in the Acceptance version, the system has to ensure that the old previous version of the script is still stored in the database (source code and compiled version) so it will not break the other environments.

Simple and usable system

As the scripting system will not only be used by Ember developers but also by customers, it is very important that the system is designed with simplicity and usability in mind. Specifically, this means having clear and easily understandable naming conventions and having well-defined instructions on how to interact with the scripting system.

Architecture and System Design

This thesis focuses entirely on the design and implementation of the new system. Therefore, I will not go into much detail about the previous scripting system that is included in the current LX product, as I simply did not require a lot of information about the previous system to design the new one. Most of the information I do have about it is from internal discussions and the Confluence documentation page. I also believe that going into extreme detail about it is not necessary since the new system is designed to be a completely new one that is not based on or an extension of the previous system.

Separation of the Scripting Framework and API

The proof of concept I created is split into four different components that come in the form of .NET projects. (The complete file structure is visualized in the [tree.txt](#) file inside my [GitHub repository](#).) Additionally, there are also two more .NET projects that focus on the user interface, as they only add a front-end and do not provide any important back-end logic, I will not discuss them in detail.

The scripting project (Scripting Framework)

The first and most important component is the scripting project, it contains the core logic that is necessary to run the scripting system, and most importantly, it also defines the necessary concepts that are later expanded on in the Ember project.

Core logic that is written in the scripting project includes the code that is used to compile scripts, execute scripts, upgrade results, and downgrade contexts. Core concepts that are defined are, for example, the numerous attributes that are used to later decorate, for example, new script versions.

Following the Clean Architecture principle, the scripting project acts as the lowest project layer of my proof of concept, it does not have access to any files inside any of the other projects and therefore is protected in the case of refactors and/or changes in the higher-level projects such as the Ember project.

To ensure better consistency, I will therefore from now on refer to this layer of my system as the Scripting Framework or the scripting class library.

The Ember project

The Ember project is designed to mock the real Ember application that will later use the scripting framework. It includes the mock internal classes I used to write test scripts, such as the LabOrder class, for example, and most importantly, it implements the abstract classes and interfaces defined within the Scripting Framework (e.g., the IConditionScript interface). To access the code written inside the scripting project, I added a simple ProjectReference to the Ember.csproj, the Ember project therefore acts as the second project layer of my proof of concept.

To ensure better consistency, I will therefore from now on refer to this layer of my system as the Scripting API. By design, this means the Scripting API also includes the layer below it, such as the Scripting Framework.

The sandbox project

The sandbox project's main purpose was to help me write scripts that I can use to test my system. In the real Ember application, such a sandbox project would be unnecessary, as the scripts are not written inside a C# file but by the user in the GUI of the Ember application where they are parsed to a string and then inserted directly into the database. Nevertheless, it was crucial to be able to isolate scripts within a dedicated .NET project during the development phase of my system, as this approach enabled native IDE capabilities such as compile-time validation, IntelliSense, and automatic refactoring that would not be possible if the scripts were simply hardcoded as raw strings within the tests.

The sandbox.Tests project

The sandbox.Tests project is where I wrote all the integration tests that validate and demonstrate the behavior of the entire system. It includes, for example, the Crud.cs file that I used to showcase how the scripting system can and should be used. It does not add more functionality to the scripting system but only security and transparency (through demonstrations).

To access the code within the inner layers (scripting, Ember, and so on), I simply added project references inside the sandbox.Tests.csproj file that point to the respective projects. Additionally, I added to each .csproj file an XML tag that enables the sandbox.Tests project to see the elements that are defined as internal within the respective project. This is extremely helpful because it allows, for example, defining classes such as the ScriptCompiler as internal (which is similar to private but allows visibility from within the assembly) and still being able to test them easily from within the Test project without having to set them to public.

The BlazorUI and BlazorUI.Tests Project

The BlazorUI project provides a web user interface that was created quickly by using artificial intelligence. The purpose of this project is to demonstrate that a web UI using the Blazor framework is not only possible but also simple and effective. The project includes an implementation of the Monaco Editor, which allows for important features such as real-time syntax highlighting and detailed compiler error messages. The purpose of the additional BlazorUI.Tests project is to use the testing library bUnit to be able to write unit tests for the Blazor components within the BlazorUI project. As the user interface was not a major objective of the bachelor project, I preferred not to invest a lot of time into writing tests, therefore this .NET project is very empty.

Architectural Principles

The new system is strongly based on multiple programming principles such as design patterns, Clean Architecture and Domain Driven Design. This is essential to ensure the system is future-proof and easily adaptable to the future needs of Ember.

Domain Driven Design:

The main idea behind Domain-Driven Design is to model a software system to match a specific business domain and its concepts, rules, and language. This should result in the code being a reflection on how the business works in real life (Evans E. , 2004, p. 13) (Fowler, Domain Driven Design, 2020). This concept was applied mainly in the beginning when I created the internal classes that the scripts are based on. One good example of this is the LabOrder class that is a direct example from their previous system LX, the version that is integrated into my proof of concept is a simplified stripped-down version that only serves for showcasing and writing scripts. The scripts written within my PoC serve mainly for testing purposes. The internal classes defined within my PoC are based on information I gathered from the Confluence (documentation) page, where I also retrieved a bunch of examples for possible future scripts that were provided to me by my employer.

Clean Architecture:

Clean Architecture is the principle of designing your system in so-called layers where the innermost layer is the most important one and is completely ignorant about the other layers surrounding it. An example in my system would be the ScriptCompiler that uses a list of references (to C# classes) and the source code of the script to create the array of bytes (the compiled script) that is then saved to the database.

The layers that surround this inner layer are dependent on it and access it from time to time to execute their operations. This separation serves to ensure changing an outer layer can only break it and possibly the layers surrounding it but never the layers beneath it. An example for an outer layer would be the ScriptManager or a customer script, changing the ScriptManager, which is basically a façade of the entire system exposing methods such as ExecuteScript, should never result in the ScriptCompiler breaking, as the ScriptCompiler is in a layer beneath the ScriptManager (Rudraksh, 2023) (Martin, 2012).



Figure 1: Visualization of the different layers of the system components.

Design Patterns Used:

Façade:

The façade pattern is one of the most relevant design patterns in software development, Gamma Erich defined the goal of it to "Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use" (Gamma, 1995, p. 184).

The façade pattern is used multiple times across the scripting system. The main spots where it is used are the Script Managers which serve as an abstraction layer (a facade) that simplifies the access to the more complex classes in the deeper layers of the system, such as the ScriptCompiler, the ScriptExecutor or the ScriptRepository.

A concrete example from my system would be the ScriptManagerBase class, which implements the IScriptManagerBase interface. It acts as the only publicly accessible API that can be called from within Ember to access the underlying scripting class library. It basically shields the rest of Ember from having to deal with the complexity of the underlying components such as the compiler and executor. Another use case for the script managers is orchestrating the interactions between some of these classes, an example would be when the developer calls the ExecuteScript method, the ScriptManagerBase uses the ScriptRepository to retrieve the compiled script and then proceeds to use the ScriptExecutor to execute it.

Another use case of the façade pattern was defining the access to the internal classes (like LabOrder), instead of directly accessing the classes, I defined interfaces that allow for a stable isolated contract between the customer scripts and the scripting API.

Repository:

The repository pattern is used to create a form of a middleman between the business logic of your application and the logic of the storage or database (Fowler, Repository, n.d.). In the case of my scripting proof of concept, the business logic is my C# code, the database logic is the Entity Framework (EF) Core code, and the middleman is the ScriptRepository class that uses the so-called ScriptDbContext to define all important operations, such as, for example, the CRUD (Create, Read, Update, Delete) operations.

The pattern was used specifically to allow the remaining system to not have to know how EF Core works (by abstracting it) and how to query data from the database.

Factory:

The factory design pattern is a pattern that handles how objects are created, most of the time it is used in places where the standard creating of objects using the new keyword and constructors is complicated, filled with boilerplate, and/or repetitive.

The factory design pattern was used in multiple places in my system. Firstly, it was used to let the future Ember developer easily instantiate the newest version of a specific context using a context factory, furthermore it was used in the demonstration of the possible future [ActionResult](#) which is a return type of the so-called ActionScript. The constructor of the class is private, and the class exposes static factory methods such as Success or Failure that both return an ActionResult object.

In both cases the factory design pattern serves firstly to centralize the creation of the objects, which is very practical because it allows one to change the entire process by only modifying one place in the codebase, and secondly to allow for loose coupling between the main code and the class that it is creating the object from (Gamma, 1995, pp. 107-116) (GeeksForGeeks, 2026).

Adapter Pattern:

An adapter (also known as a wrapper) acts as a bridge between two incompatible interfaces so they can work together. Gamma Erich defined its intend as being to "Convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces"

The adapter pattern was used primarily in the versioning system of my proof of concept. Firstly it is used in the ActionResult classes, where each version (class) inherits from the abstract class called ActionResultBase that implements the IUpgradeableReturnValue interface, which forces each ActionResult to implement a method called "Upgrade" that takes in the previous version as a parameter and uses it to map its properties to a new object of the class that contains the Upgrade method.

Furthermore, the pattern is also used similarly when performing context downgrades, the concept of context downgrades will be discussed more in depth in the Implementation chapter.

Both instances where the pattern was used were to ensure backwards compatibility when executing scripts that still might rely on older contexts or return older versions of an ActionResult, the system therefore adapts (downgrades the context, for example) iteratively until it reaches the desired version or until it throws an exception if it tries to adapt to a version that is no longer supported. (Gamma, 1995, p. 139)

Strategy Pattern:

The strategy pattern lets you define a so-called “family” of algorithms or behaviors where you put each of these algorithms in its own class and make them interchangeable at runtime.

It allows for better flexibility at runtime because it lets the algorithm vary independently from the client that uses it (w3sDesign, 2018, p. 268).

The strategy pattern was crucial to allow the system to be abstract and allow for easy future enhancements or changes. One good example are the numerous base interfaces I defined, such as the IConditionScript and the IActionScript which are marker interfaces (empty interfaces) that implement the IScriptType (which is a deeper nested strategy) interface, they are used to decorate the interfaces of different script types to be able to differentiate between them. This approach allows for the different components of the system to treat the different types of script the same way, as if they were the same exact type.

Dependency Injection:

Dependency injection is the idea of separating the process of object creation from the application, this is often achieved by giving an object the other objects (also called dependencies) rather than having the object create them itself.

In .NET this is done by adding the services (dependencies) into a so-called dependency injection container that then injects them into the constructor of the object that we want to create.

Dependency Injection is useful because it allows for easier testing through the ability to mock objects and because it allows loose coupling between the main code and the object creation process (w3sDesign, 2018, p. 322) (Ulelu, 2024).

Using dependency injection was a big requirement set by my host-company, it is used mainly to register dependencies such as the ScriptCompiler, the ScriptRepository, and the Logger. Later on, it can then be used to for example inject the ScriptManager like I did it in the InitScriptManager method within the [TestHelper](#).

System Components

Compilation Strategy

The Compilation Problem:

Since in the new system we are not writing scripts in an interpreted language such as Python anymore but in C#, which is a compiled language, it introduces the problem of scripts needing to

be compiled before they can be executed. This then raises the question when scripts are compiled and where and how the compiled outputs should be stored. There are multiple possible approaches that could have been chosen, with each of them having pros and cons.

The main challenge in choosing one of them is therefore to balance performance and flexibility while also ensuring a user gets immediate feedback if there is something wrong with his code, for example, it would be very beneficial to the user right after having written a script to know if it contains any obvious errors.

Possible Solutions: Using C# compiler Roslyn

Strategy 1: Just in time Execution

Compile on execution, also called JIT (just-in-time) compilation is the most obvious and also simplest solution. It works by taking the source code and compiling it to a byte array that is immediately discarded once the script was executed only once.

It has the advantage of being simple and fast to develop, and it avoids having to store the script source code and the compiled script byte array separately and therefore saves some disk storage.

The disadvantages are that this technique is extremely resource wasteful, since we are always recompiling the script source code for each execution, which is much more expensive than querying a saved compiled script from a database.

Considering this notable performance hit, it would very likely be too slow for production use in the future Ember LIMS. Furthermore, considering that scripts are written once and modified very rarely, it is worth investing the extra time and complexity into using an approach that does not rely on JIT compilation (Teru, 2023).

Strategy 2: Ahead of time compilation

Ahead-of-time compilation, also called AoT compilation, works by compiling the script before it is ever called for execution. This pre-compilation can be done at different points in time, although the simplest and arguably best solution is to just compile it whenever the user creates the script or modifies it.

The implementation in my system also uses this approach. The component responsible for transforming the source code into an array of bytes is the ScriptCompiler. Following that, the array of bytes is then saved to the database. Using this approach means the source code of the script needs to be stored separately in a different table.

Using this approach comes also with some drawbacks since the compiled scripts take up extra space on the database, which will lead to higher memory usage on the server that is running the database for Ember. Although this is negligible, as these compiled scripts still do not take up a large amount of memory that will have any impact on an enterprise-grade server running a LIMS such as Ember (Teru, 2023).

Storage and Caching

The database I used to store the scripts and the compiled scripts was PostgreSQL, it was chosen by my host company for its strong consistency and clean data model. Although PostgreSQL is fast when it comes to querying for scripts, it can still lead to a bottleneck in the program flow in situations where a specific script is executed in a high-frequency code path, such as, for example, in a loop. To speed up such use cases, one could implement in the future some sort of high-speed cache that stores scripts that were recently queried for.

This high-speed cache could be realized by implementing a secondary Redis database that is checked before the main PostgreSQL database. If the Redis cache does not contain the requested script, it will just fall back to the PostgreSQL database and retrieve it from there. For a caching algorithm, one could design a complex caching algorithm or simply choose the simplest option, which just consists of caching the N most recently queried scripts, where N is a number chosen based on the amount of RAM available.

Implementation

Script Compilation and Validation

A compiler is a piece of computer software that takes in human-readable source code (e.g., customer script) from a high-level programming language such as C# and translates it into machine language. This machine language can then be used by the computer to execute the script (compiler, 2026). To be specific, in the PoC instead of directly compiling to machine language, the script is first compiled to intermediate language (a .NET assembly) by the .NET compiler platform, which is also known as Roslyn, and then later on compiled to machine code by the runtime.

The use of Roslyn was absolutely crucial in the creation of my system, as Roslyn is not a simple “source-code-in and object-code-out translator” but a complete API that one can use to perform complex tasks on source code and get in-depth information about it. I therefore used Roslyn extensively in my `ScriptCompiler` class primarily because it allowed me to compile and analyze the source code of the scripts, all within the host application (Ember) (The .NET Compiler Platform SDK, 2026).

Script Compilation

The [ScriptCompiler](#) class is the component responsible for both the compilation and validation of the scripts. The `RunCompilation` method is responsible for the compilation process. To compile a script, it begins by parsing the string into a so-called `SyntaxTree` which represents the hierarchical structure of the source code.

To create the desired `CSharpCompilation` object, we need to pass the `SyntaxTree` alongside a list of `MetadataReference` objects, this list is a mix of references to standard .Net assemblies such as `System.Runtime` for example, and most importantly, references to the assemblies containing the scripting API (in our case, references to Ember). These references are passed to the `ScriptCompiler` once during the start-up of the application using dependency injection inside the [ScriptingServiceCollectionExtensions](#) class.

After having created the `CSharpCompilation` object using the references and the `SyntaxTree`, it calls the resource-intensive `Emit` method that executes the complete compilation process and writes the compiled intermediate language to a `MemoryStream`. Following that I simply check the `Success` property on the result, and if it is true, I return the array of bytes from the `MemoryStream` (Woodward, 2016).

Script Validation

The validation of scripts is very important for a few reasons.

1. If the validation fails, it prevents an unnecessary expensive compilation process.
2. We can define custom rules for script which allows for more control over what scripts are allowed to be inserted into the database.
3. It allows for more detailed exceptions to be thrown.
4. It allows me to extract metadata from the scripts source code such as the name , script type and Context version of the script.

The BasicValidationBeforeCompiling method

In the ScriptCompiler class I am performing multiple validations inside multiple different methods.

The BasicValidationBeforeCompiling method is the most relevant one, as it internally calls all the other validation methods. Its purpose is to validate the script's source code and throw the correct exceptions according to what went wrong, it also serves to return a ValidationRecord that contains metadata about the script that is used within the ScriptRepository to create the correct database record.

It works similarly to the RunCompilation method in the sense that it uses the same references combined with a SyntaxTree to create a CSharpCompilation. It is important to note that the creation of this Compilation object is not the resource-intensive part of the compilation process and therefore is fine to be executed inside the validation method.

After creating the CSharpCompilation object, I retrieve a SemanticModel, which I can then use to analyze the script's source code. For example, I am using the model to check how many classes are defined in the source code so that I can enforce the script to only define one class within its source code.

After retrieving the model, I can use it to retrieve the implemented interface of the class of the script, which I called parentSymbol. This parentSymbol is then used within the GetVersionInt method to retrieve the version of the script and later the type of the script (e.g., GeneratorConditionScript).

Checks done inside the Basic BasicValidationBeforeCompiling method:

Validates	Throws
The script string cannot be null or empty.	ScriptWasEmptyOrNullException
Only one class per script source code.	MoreThanOneClassFoundInScriptException
The implemented interface needs to be valid so that a version can be assigned to the script.	VersionIntNotAssignedException

The ValidateScriptMethodTypes method

The ValidateScriptMethodTypes method serves to validate that a script can only define methods that are defined on the interface that it implements. This is necessary, as each method that is defined on a script version interface is optional and has a default implementation (through default interface methods) so that it won't throw a compile-time error. Therefore, it makes sense to check that each public method a script implements is actually defined on the specific interface it implements. Another check one should do is to verify a script has at least one public method, as without one there is no real functionality to the script. It is important to note that I allowed private methods to be defined, as there can be a purpose for a method that is only called within the class of the script (internal methods are also forbidden).

This method not only verifies the names of the methods but also the complete signatures, including the return type and the parameter names and types. After verifying each method, it then returns the list of the defined methods on the script, which is included in the returned metadata from the BasicValidationBeforeCompiling method.

Checks done inside the `ValidateScriptMethodTypes` method:

Validates	Throws
The public script methods name can't be matched to a method defined on the interface.	<code>UndefinedMethodException</code>
Matched methods return type is different to the one defined inside the script.	<code>WrongReturnTypeErrorException</code>
The matched method has a parameter not defined inside the interface method.	<code>CouldNotFindParameterException</code>
A parameter in the matched method had the wrong type.	<code>WrongParameterTypeErrorException</code>
There was no public method defined inside the script.	<code>NoMethodInScriptException</code>

The `GetExecutionTime` method

The `GetExecutionTime` method is used to extract an optional attribute that can be defined above the class that is defined inside the script. It serves to give the customer the ability to define how much time a script gets when it is being executed. This method, unlike the others, does not throw an exception as it serves merely to extract the optional execution time information from the script.

```
[ExecutionTime(ExecutionTimeGroups.Long)]
public class ExecutionTimeScript :
//regular script definition ....
```

Note: Example of the usage of the optional `ExecutionTime` Attribute

The `ValidateLoopsHavingCancellation` method

The `ValidateLoopsHavingCancellation` method is necessary to ensure that each loop within the script checks the predefined Cancellation token at least once in its body (so each iteration), this is absolutely necessary because in .NET it is not possible to forcefully abort a thread anymore.

It is important to note that this solution is flawed and should be iterated in the future, as it is still possible for customers to circumvent the cancellation token being called in each iteration by

writing tricky code (for example, using a goto statement). This could lead to a CPU resource leak as the core will run the script infinitely.

The `ValidateNamespaceUsage` method

Because scripts are not sandboxed but run inside the same application domain as the primary software system Ember and therefore also have the same permissions and access rights, it is particularly risky to not restrict what libraries scripts can access. To restrict the access to specific libraries (stored inside an array of strings), I created this method that traverses both the `UsingDirectiveSyntax` (the usings statements at the top of a C# file) and the `IdentifierNameSyntax` (for example, when a customer explicitly writes `System.IO.File.ReadAllText`).

Forbidden Namespace	Security Risk
System.IO	File System Access
System.Net System.Net.Http	Network Access
System.Reflection	Encapsulation Bypass
System.Diagnostics	Process Manipulation
System.Runtime.InteropServices	Unmanaged Code Execution
System.Threading.Thread System.Threading.ThreadPool System.Threading.Timer System.Threading.Mutex System.Threading.Semaphore System.Threading.SemaphoreSlim	Resource Exhaustion & Deadlocks

Note: This table and also the List of banned namespaces in the source code was generated via AI, I simply asked Gemini what namespaces should be banned and why they should be banned and then created this Table from the answer (Edgerson, 2025) (Security Considerations for Reflection, 2026) (Security rules, 2026).

The `ValidateAllowOnlyRecentTypes` method

The idea behind the static class `AllowOnlyRecentTypes` and the method inside of it is to only allow the customers to insert scripts that are of the most recent version, the method therefore

checks that the parentSymbol (the interface implemented by the script) is inside of the list of recent types passed to the ScriptCompiler via dependency injection. In retrospect, I have to admit, relying on dependency injection to get this list of recent types creates an additional unnecessary configuration. A cleaner and safer solution for a future improvement would be to use reflection to perform an assembly scan at startup, allowing dynamic discovery of the latest versions of the interfaces and classes without requiring manual configuration of this within the dependency injection.

Script Execution

After a script was successfully validated and compiled, it can be executed from within the environment of the Ember application. The complete process of execution is managed by the ScriptExecutor class, which handles loading the array of bytes (the compiled script) into the memory of the system, instantiates the script class, invokes the desired method, and manages the time a script gets to execute (execution timeout).

The entire execution process is happening in an asynchronous method that returns a Task<object> called RunScriptExecution, the first step of the execution process is converting the array of bytes back into an executable assembly using the Load method. In .NET, loading an assembly using reflection allows me to dynamically create types and invoke methods directly within the host application. This is the crucial concept that makes the entire scripting system possible.

Once the assembly is loaded, the ScriptExecutor scans the assembly for exactly one class that implements an interface that inherits from IScriptVersion. If this step was successful, the executor creates an instance of the retrieved class.

With the script instantiated, the executor can now, using reflection once again, locate the desired method and invoke it while also passing the context the method needs to access data within the Ember application. Finally, the script is executed asynchronously using the WaitAsync method.

```
Assembly assembly = Assembly.Load(compiledScript); //where compiledScript is of type byte[]
Type type = assembly.GetType()
    .Single(t => t.IsClass && !t.IsAbstract && typeof(IScriptVersion).IsAssignableFrom(t));
object scriptInstance = Activator.CreateInstance(type!);
MethodInfo method = type.GetMethod(methodName!);

Task? resultTask = (Task)method.Invoke(scriptInstance, new object[] { context });
await resultTask.WaitAsync(_scriptTimeout);

var resultProperty = resultTask.GetType().GetProperty("Result");
var resultValue = resultProperty!.GetValue(resultTask);
```

The entire [RunScriptExecution](#) method can be summarized like this.

Script timeouts

As it is impossible to write perfect bug-free scripts and validation methods, being able to control the time a script gets to execute is a crucial part of having a safe scripting system. This is especially true, as in the context of Ember, the scripts are written not by the Ember developers but by customers, and it is practically inevitable that one day someone will accidentally write a script that, for example, has an infinite loop. To prevent such a script from being able to permanently block the complete main application, it is necessary that we have the ability to terminate the execution at any time.

The initial iteration of the `ScriptExecutor` relied entirely on the `WaitAsync(_scriptTimeout)` method, this seemed to work perfectly initially, as it successfully throws a `TimeoutException` (after `_scriptTimeout` milliseconds) that is being caught by the `ScriptExecutor`. After some time, though, I was made aware by my manager that using this approach, the task is never notified about the main thread not waiting for it anymore and therefore is never truly aborted. In the case of an infinite loop inside the script (such as `while(true)`, for example), this would lead to an orphaned task that can't be terminated anymore from inside Ember. In the worst case this or a similar faulty script could be triggered multiple times, which would lead to a massive loss in performance as the Ember application keeps running because each script would take up a thread permanently, and the system would quickly suffer from thread-pool starvation.

The easiest way to solve this problem would have been to use the `Thread.Abort()` method, but this mechanism is deprecated in modern .NET 9, as Microsoft intentionally only allows for "cooperative cancellation" Forcefully terminating a task in the middle of execution without proper handling was removed from .NET because it can lead to the system being in an invalid/corrupt state, therefore Microsoft enforces tasks being cancelled using the so-called Cancellation token which in our example means that the running script needs to periodically check the state of the token. If it turns out that a cancellation was requested by Ember, the script will gracefully abort without the risk of the system ending up in a corrupt state. Specifically, this means that in each customer script the state of the cancellation token needs to be checked in every loop at least once. It is important to note that there were options for forceful termination, such as process termination or an unsafe method called `ControlledExecution.Run`, those were not chosen because of a massive performance overhead or because they are simply unsafe (Macam, 2020) (Cancellation in Managed Threads, 2026) (Task cancellation, 2026) (`ControlledExecution.Run(Action, CancellationToken)` Method, 2026).

```
using var cts = new CancellationTokenSource(scriptTimeout);
ScriptEnvironment.CurrentToken.Value = cts.Token;
await resultTask.WaitAsync(cts.Token);
ScriptEnvironment.CurrentToken.Value = CancellationToken.None;
```

These are the primary changes and additions to the previous code-snippet that allow for the usage of a CancellationToken.

ScriptRepository and ScriptManagerBase

The `ScriptRepository` class, which implements the Repository design pattern, uses Entity Framework Core alongside an `IDbContextFactory`. This ensures thread-safe and scoped database operations for storing `CustomerScript` objects (scripts alongside their metadata) and their respective `CompiledScript` objects (compiled scripts consist of a byte array and metadata). This architecture of isolating the EF Core code inside of the `ScriptRepository` class prevents the query logic from leaking to higher levels of the system, which would violate the Clean Architecture principles.

Beyond the standard database operations, the class also includes several methods that facilitate data retrieval, such as, for example, a method called `GetAllCustomerScripts` that takes in an optional `CustomerScriptFilter`.

The `ScriptManagerBase`, which is an implementation of the façade design pattern, serves to simplify the access to the more complex underlying classes, such as for example, the `ScriptRepository`. It does this by exposing a set of public methods that orchestrate the actions between the compiler, executor, and repository. These methods then act as a public API while keeping the entire system more modular, testable, and loosely coupled.

The ScriptManager

The `ScriptManager` acts as a higher-level abstraction compared to the `ScriptManagerBase`. It is designed to simplify the usage of the complete scripting system by ensuring the developer works exclusively with the most recent versions of scripts, contexts, and custom return types. While the API only accepts and returns these modern types, the `ScriptManager` preserves backwards compatibility behind the scenes by automatically employing a context downgrade strategy before execution and a custom return type upgrade strategy after a script successfully completed. Furthermore, it seamlessly integrates script facades, which act as proxies for the actual scripts by implementing the exact same interfaces. In retrospect, it is clear that during the development phase I confused the façade pattern with the proxy pattern, as these “script facades” should be called script proxies.

Overview of core components and their responsibilities

Component	Architectural Pattern	Primary Responsibility	Key Injected Dependencies
ScriptRepository	Repository	Managing the state of the database and executing queries using EFCore	IDbContextFactory, ScriptCompiler, IUserSession
ScriptManagerBase	Facade	Providing a clean API for advanced operations	ScriptRepository, ScriptCompiler, ScriptExecutor
ScriptManager	Facade	Provides a clean and simple API for standard operations	ScriptManagerBase

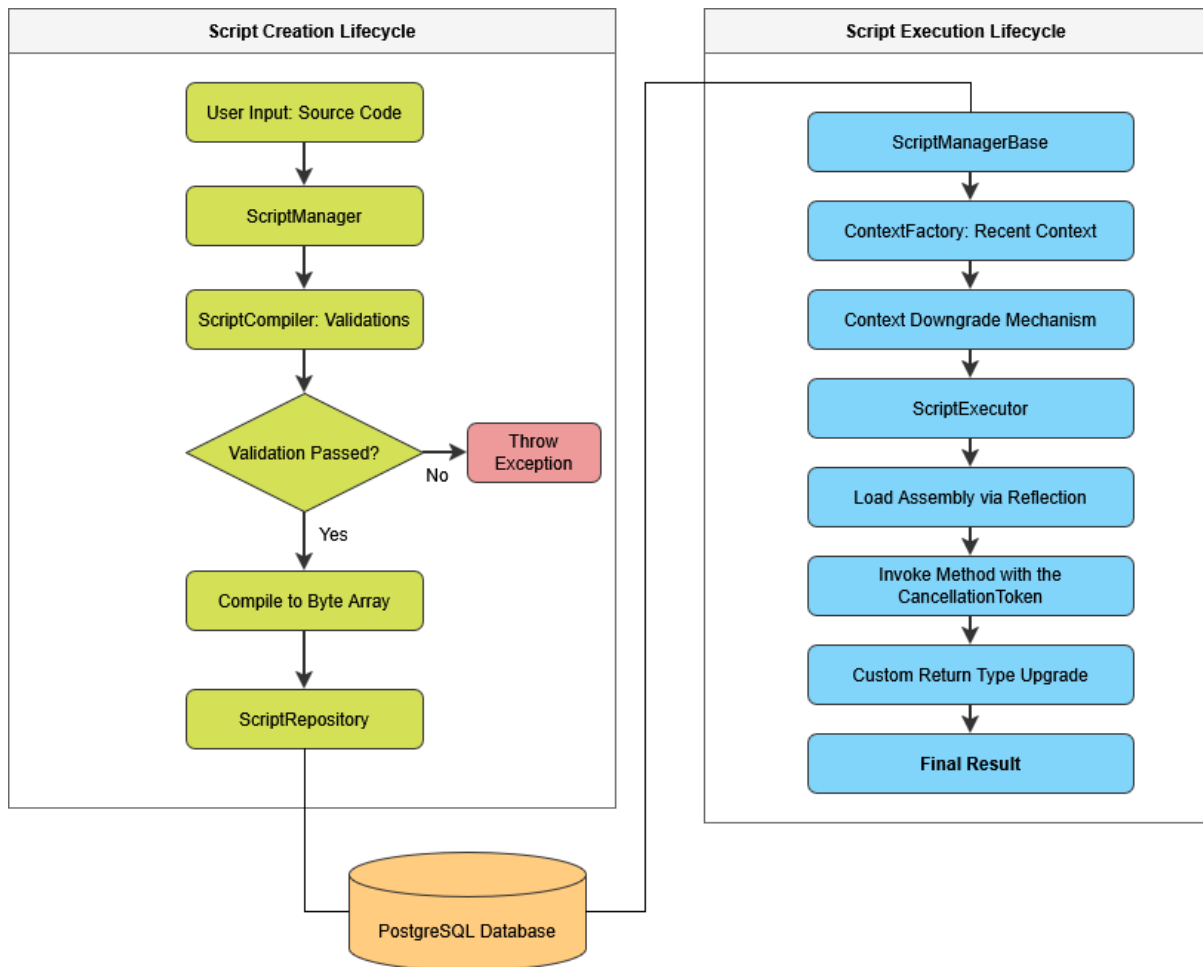


Figure 2: Final lifecycle of the Script Creation and Execution

The scripting API in detail

To achieve the goal of a simple and capable scripting system, a clear structure of how the scripts are defined and how they should look was needed, over the course of the 12-week internship period, I explored multiple different script designs.

How scripts are defined:

Each script is defined by a class with the name of the script being the name of the class, initially the system design was focused around each script having exactly one method that is pre-defined by the developer inside Ember. At first, the concept of having one entry method per script seemed to be fine, as it seemed to be a simple and robust solution for the Ember requirements, but as I moved forward with the proof of concept, it became clear that it would be more practical for the future customers to be able to define more than one method per script.

Allowing multiple methods in a script comes with multiple challenges, as the scripting framework, specifically the **ScriptExecutor**, has to know the specific method that it should

execute on the instance of the script (instance of the class of the script). To solve this problem, I decided to use a similar approach as before and let the customer only define methods that the Ember developer has already pre-defined within Ember.

```
public class SetCustomFieldScript : IActionScriptV1
{
    public async Task<ActionResult> ExecuteAsync(IContext context)
    {
        context.LabOrder.SetCustomField("ProcessedByGenerator", true);
        return ActionResult.Success("Pediatric tests added V3");
    }
}
```

Example code of how a customer script could look, it is simplified example not included in my system

Differentiating between types of scripts

Early in the design process, my managers provided me the idea of having 2 different types of scripts, a script type called ConditionScript that has read-only access to the internal Ember classes and returns a Boolean and a so-called ActionScript that has read and write access to the internal classes in Ember and returns an object (or subtype) of a class called ActionResult.

This idea of having only two types of scripts was abandoned pretty early as it became clear the system could be much more modular and customizable in the future if it allowed the Ember developers to define their own types of scripts with their own permissions and return types.

Even though the concept of having two types of scripts was abandoned, there are still a lot of places where these two types are referenced, as they are good examples of possible script types that could be implemented in the future.

To define these different script types that the customer will have to use, I created a marker interface called IScriptType inside the scripting framework that is then inherited from by another marker interface, such as for example, the [IConditionScript](#) interface that is located within Ember.

To separate the types of the scripts from the actual versions of the scripts, I created another marker interface within the scripting framework called IScriptVersion that is then inherited by the actual interfaces that define the scripts and their properties, such as allowed methods, return types, and most importantly, the type of context that is passed to the methods.

```
public interface IActionScriptV1 : IScriptVersion, IActionScript
{
    Task<ActionResult> ExecuteAsync(IContext context)
    {
        throw new NotImplementedException();
    }
}
```

Example code of how a script interface version could look, it is simplified example not included in my system

The script contexts

To give customer scripts the power to read and maybe perform actions on the running Ember instance, we need to pass them the according objects that they need to perform the desired actions. This could have been solved by tediously passing a list of objects into the parameters of a script method or, like it was solved within the PoC, by utilizing the Context Object Pattern and creating an abstraction called "context" that contains all the objects that would have been passed as parameters to the script.

The added layer of abstraction created by the implementation of the IContext interface allows the developers to have more fine-tuned control over what scripts can access. Specifically, it allows developers to define specific types of contexts that are used on specific types of scripts, for example, one could create an IActionScriptContext that can only be passed to methods inside of an ActionScript. As the script interfaces will be versioned, it is only logical to also allow the contexts to be versioned, this versioning will be discussed in the later chapter.

The Versioning System

Script Types

As already mentioned, script interfaces need to support versioning. The original design planned to implement this using inheritance, as it seemed the most logical for this kind of requirement, but after some deeper analysis of the requirements, it became clear that using inheritance to version the script interfaces would force new versions that inherit from the previous ones to include old, deleted, or renamed methods, but since we want to allow developers to make deliberate, sometimes breaking changes to the API, using inheritance is not an option.

Instead, an attribute called "MetadataScript" was created that has to be placed above the interface that represents the script version. This approach, of course is not compile-time safe but can easily be checked by implementing Automated Architectural Testing, for example, one could write unit tests that check for duplicate versions on an interface with the same IScriptType or

interfaces that implement both `IScriptVersion` and a subtype of `IScriptType` while not defining the required attribute.

Naming the script interface versions differently depending on the version (e.g., `GeneratorScriptV4`) could lead to major confusion on the side of the customer. It is also clear that the customer should not know or care about what version he is currently handling while writing his script. Therefore, I decided to put each version of a script interface into its own namespace with the according name, this will then allow the future Ember script editor to just automatically import the current namespace on the top while allowing the user to just implement the same interface name for each version (e.g. `IActionScript`).

```
namespace GeneratorScriptV2
{
    [MetaDataIScript(version: 2)]
    public interface IActionScript : IScriptVersion
    , Ember.Sandbox.ScriptingFrameWork.ScriptTypes.IActionScript
    {
        //removed the methods for simplicity...
    }
}
```

Example of script interface version from my system, I removed the methods for simplicity

To prevent the Ember developers from constantly having to interact with and manage different versions, which would add a considerable amount of complexity that could lead to errors being made down the road. The `ScriptManager` class was added to the system, which allows the developers to always interact with the latest version of a given type of script. To achieve this, I created the `IRecentScriptFacadeInterface` which is defined inside the scripting framework. This interface can then be implemented alongside the recent script type interface (e.g., `IGeneratorScriptV3`) by a class with the name of the type of the script, for example, `ActionScript`.

To allow the developers to now use these new `IRecentScriptFacadeInterface` instances, I modified the `ScriptManager` methods (specifically the `Get` and `Create` methods) to only return and accept instances of this interface.

These “script facades” are an implementation of the proxy pattern, there purpose is simulating an instance of the script. This is possible because the facades implement the same interface that the script implements, which leads to the façade exposing the same public methods as the script. Therefore, it is possible for the developers to simply use the `ScriptManager` to `Get<ScriptFacade>(name)` the desired script and then call the desired method on the returned object, just like it was the actual script that was returned, while the façade takes care of the actual instantiation of the script using the `ScriptManager` and `ScriptManagerBase`.

This approach of course comes with some downsides:

1. A script of an older version might have a method that has been renamed since.

In this rare case instead of immediately calling `ExecuteScript`, the Façade adapts the name of the method depending on the API version of the script.

2. The current version might not support some older methods that are implemented on the script.

In this case those lacking methods were intentionally deprecated and cannot be called anymore by design, the script should therefore be updated as soon as possible.

3. Since all methods are optional, the façade likely exposes methods that are not implemented on the actual script being called.

Since the person working with this script façade is the Ember developer, he should be aware of what methods are used in which script, if he accidentally calls a method that is not defined on a script, the system throws a `CouldNotFindMethodException`.

4. The current version, which the façade exposes, might have new methods not defined on the old script.

Similarly to the previous case, the Ember developer should know which methods are defined on the script he is working on, if he makes a mistake, the same `CouldNotFindMethodException` is thrown at runtime.

Contexts

Since over the course of Ember's lifetime script interfaces will be versioned, it is also necessary to allow the contexts to evolve alongside them. To ensure the scripting system is robust and does not break older script's a versioning mechanism was designed based on three key concepts, a versioned interface hierarchy, a runtime context downgrade mechanism, and a reflection-based version scanner.

The Context Interface Hierarchy

The entire versioning system is based on three different marker interfaces that are defined inside of the scripting library:

1. `IContext`:

This is the base interface that is shared among all the context types.

2. `IDowngradeableContext`

This interface, which is a subtype of `IContext`, declares that a context is capable of being downgraded to a previous version using the `Downgrade` method.

3. `IRecentContext`

This interface is designed to mark the most recent version of a type of context, for example, the most recent version of the `GeneratorContext` is version 5.

```
public interface IContext { }

public interface IDowngradeableContext : IContext
{
    public IDowngradeableContext Downgrade();
}

public interface IRecentContext : IContext { }
```

Context version interfaces

Defining a new context inside the scripting system is a multi-step process, to demonstrate how it should be done I will use the example of the so called `GeneratorContexts`.

To implement this specific type of context that should then ideally only be used by a specific type of script such as `GeneratorScripts`, one first has to define a marker/base interface that then has to be inherited from every version (in the form of an interface) of that type of context, the base/marker interface is also the only interface that defines if this specific type of context is a downgradable one or a regular (non-downgradable) one.

```
[MetaDataIntrfc(version: 0, type: TypeInfo.AbstractBaseInSF)]
public interface IGeneratorContextBaseInterface : IDowngradeableContext { }
```

Example of such a marker interface that is then later used to mark contexts as `GeneratorContexts`

The second step in creating a new context is defining the interface for it, this can be done in multiple ways depending on the situation. If the desired context should be the first version of this type of context, one simply inherits the before-mentioned base/marker interface and defines the desired properties that the script should be able to access.

If, on the other hand, the desired context is not the first version but a new version of an already existing one, one can choose between creating a new version from scratch by again implementing the before mentioned base interface or one can choose to use inheritance to inherit from the previous version.

```
namespace IGeneratorContextNoInheritance_V5
{
    [MetaDataIntrfc(version: 5)]
    public interface IGeneratorContext :
        IGeneratorContextBaseInterface, IRecentContext
    {
        ILabOrderInterfaceV4NoInheritance LabOrder { get; }
        IVaccineInterface Vaccine { get; }
    }
}
```

New version of a context starting blank.

```
namespace IGeneratorContext_V4
{
    [MetaDataIntrfc(version: 4)]
    public interface IGeneratorContext : IGeneratorContext_V3.IGeneratorContext
    {
        new ILabOrderInterfaceV3 LabOrder { get; }
    }
}
```

New version of a context using inheritance.

Context version class

The third and final step to create a context is then to simply create a class that inherits the before-defined context version interface.

```
namespace GeneratorContextNoInherVaccineV5
{
    [MetaDataClass(version: 5)]
    public class GeneratorContext :
        IGeneratorContextNoInheritance_V5.IGeneratorContext
    {
        // code...
    }
}
```

Implementation of a context class.

As seen in the previous code examples, each context class and interface is put in a different namespace to enable calling each context the same name. This is essential as it enables the end-user (customer) to always use the same name when writing his scripts (for example GeneratorContext), as the correct namespace can be automatically imported when a new script is written.

The Runtime Context Downgrade Mechanism

Since avoiding that the developers have to worry about which context version to instantiate and pass to the methods is a crucial requirement of the system, a context factory was created that simply creates the most recent version of the requested context type.

Only allowing the developer to instantiate the most recent version creates a compatibility issue where older scripts that are still in the database and have not been updated yet might rely on older versions of the context. To solve this problem, I created the ScriptManager which acts as a middleman between the developers input and the methods within the ScriptManagerBase, this middleman then takes care of taking the recent version of the context that was created by the factory and transforming it into the correct context that the script requires.

This process of transforming a new context into an old context is called the downgrade mechanism, it works by having the IDowngradableInterface be implemented on the context type interface (for example IGeneratorContext), this then forces the subsequent Context interface versions to implement a downgrade method that takes in a context of their own type and constructs a context of the previous version. These downgrade methods will, of course be written by the Ember developers and therefore allow for a maximum amount of control over how and if a context is downgraded.

```
public IDowngradeableContext Downgrade()
{
    ILabOrderInterfaceV3 labOrderV3 = (ILabOrderInterfaceV3)LabOrder;
    Patient patient = new Patient("1", "Default", "Default", new DateTime(2010, 6, 1, 7, 47, 0), "M");
    ConsoleLogger logger = new ConsoleLogger();
    DataAccess data = new DataAccess();
    return new GeneratorContextV4.GeneratorContext(labOrderV3!, patient!, logger!, data!);
}
```

Example of an implementation of the downgrade method taken from [GeneratorContextNoInherVaccineV5](#).

The context manager and the reflection-based version scanner

The actual downgrade process is defined inside the scripting class library inside the [CreateByDowngrade](#) method. It works by recursively calling the Downgrade method on the passed recent context.

To allow the ContextManager to detect that it has reached the correct version, a ContextVersionScanner was created that takes care of mapping concrete types to versions. It uses reflection to scan the entire AppDomain at runtime and finds all types (context classes in this case) that implement IContext, after that it then tries to retrieve the MetadataClass attribute that includes the version, this attribute should be defined above each context class.

Context Factory

The context creation is done using a context factory, this factory is injected via dependency injection.

```

var services = new ServiceCollection();
Ember.Simulation.SandboxServiceCollectionExtensions.AddSandboxServices
(services, logger, testDataAccess);
using var provider = services.BuildServiceProvider();
RecentGeneratorContextFactory.IGeneratorContextFactory factory = provider.
GetRequiredService<RecentGeneratorContextFactory.IGeneratorContextFactory>();

RecentIGeneratorContext ctx = factory.CreateGeneratorContext(labOrder, vaccine);

```

Example of how the developer retrieves the context factory and how he instantiates a context using the internal classes LabOrder and Vaccine

Custom Return Types

Customer scripts do not just execute and terminate but can also return a structured result object that is later inspected by Ember. Because the entire scripting system is versioned, the form of this return type should be allowed to change across API versions (the same concept as the versioned context). Therefore, the system needs to be able to take the result that is returned by a script running on an older version and transform (upgrade) it to the desired type of result that the Ember application is currently working with.

To define a new custom return type, one simply creates a new abstract class that inherits from another abstract class that is defined within the scripting library called CustomReturnType, this also enforces each custom return type to implement an Upgrade method.

```

public abstract class CustomReturnType : IUpgradeableReturnValue
{
    public abstract object Upgrade(object returnValue);
}
public interface IUpgradeableReturnValue
{
    public object Upgrade(object returnValue);
}

```

Definition within the [scripting library](#)

The purpose of the upgrade method is to take in an instance of the previous version and return an instance of the current one.

The intermediate abstract class ActionResultBase serves as a base type that is implemented by each ActionResult type class. In retrospect I realize that a marker interface should have been used instead of an empty abstract class.

```
public abstract class ActionResultBase : CustomReturnType
{
}
```

Intermediate abstract class used to decorate the following [ActionResult](#) return types.

When a new version is created, one can choose between inheriting from the previous version or implementing a completely new version by inheriting from the base abstract class (ActionResultBase).

```
namespace ActionResultV3
{
    [MetaDataCustomReturn(version: 3)]
    public class ActionResult : ActionResultBase
    {
        //code...
    }
}
```

Example of the definition of a new custom return type version without using inheritance.

Final usage of the ScriptManager

To help showcase how the scripting API should ideally be used, I created a test file called [Crud.cs](#), where I tried to showcase how the most common operations are handled.

Dependency Injection

The first step of using the scripting API is injecting the ScriptManager and ScriptManagerBase, this is demonstrated inside the InitScriptManager method at the top of the CrudDemos class that is inside the beforementioned file.

```
ServiceCollection services = new ServiceCollection();
LoggerForScripting logger = new LoggerForScripting();

services = new ServiceCollection();
services.AddLogging(builder =>
{
    builder.AddSerilog(dispose: true);
});
services.AddSingleton<IUserSession, SandboxUserSession>();
services.AddDbContextFactory<ScriptDbContext>();
ScriptingServiceCollectionExtensions.AddEmberScripting(
services, EmberMethods.GetReferences(), EmberMethods.GetEmberApiVersion(),
RecentTypeHelper.GetRecentTypes()
);

var provider = services.BuildServiceProvider();

IScriptManager scriptManager = provider.GetRequiredService<IScriptManager>();
IScriptManagerBaseExtended scriptManagerBase = provider.GetRequiredService<IScriptManagerBaseExtended>();
```

The entire process of injecting the script managers.

Create

Creating a script and adding it to the database can be done via 2 ways, the first way using the ScriptManager returns an instance of the script type façade, this instance can then be used to call

methods on the script within the database. The second option executes the same underlying code but instead returns the CustomerScript object that contains metadata such as name, source code, and time of creation. The option to return the created script was added, as it allows to reduce the needed operations done by the developer when working with scripts.

```
//To retrieve the Script façade(proxy) instance
ActionScript scriptInstance = await _scriptManager.CreateScript<ActionScript>(sourceCode);

//To retrieve the more in depth MetaData
CustomerScript script = await _scriptManagerBase!.CreateScript(sourceCode);
```

Read

Similarly to the creation of a script, there are two different methods that allow for the retrieval of a script that is inside the database.

```
//To retrieve the script façade(proxy) instance
ActionScript scriptInstance = _scriptManager.GetScript<ActionScript>("AddPediatricTestsV2");
//To retrieve the more in depth containing the byte[]
CustomerScript script = await _scriptManagerBase.GetScript<IActionScript>("AddPediatricTestsV2");
```

Update

Updating a script is done over the ScriptManagerBase, as it is a less common operation. Using the UpdateScript method, it also automatically updates the metadata of the script, meaning that the next time you query for it, you will have to look it up by its new name (only if the class name was changed of course).

```
await _scriptManagerBase.UpdateScript<IActionScript>("AddPediatricTestsV2", newSourceCode);

//Name is automatically updated according to the new source code
CustomerScript script = await _scriptManagerBase.GetScript<IActionScript>("AddPediatricTestsV4");
Assert.IsTrue(script.SourceCode == newSourceCode);
```

Delete

As deleting a script from the database is a more advanced operation it is handled using the ScriptManagerBase, via the DeleteScript<>() method.

```
await _scriptManagerBase.DeleteScript<IActionScript>("AddPediatricTestsV2");

await Assert.ThrowsExceptionAsync<System.InvalidOperationException>(async () =>
{
    CustomerScript script = await _scriptManagerBase.GetScript<IActionScript>("AddPediatricTestsV2");
});
```

Injecting the ContextFactory

Injecting the ContextFactory is an operation that is done quite rarely, as it is only needed once per service, nevertheless it is a necessary step when executing scripts.

```
ConsoleLogger logger = new ConsoleLogger();
DataAccess testDataAccess = new DataAccess();

var services = new ServiceCollection();

Ember.Simulation.SandboxServiceCollectionExtensions.AddSandboxServices
(services, logger, testDataAccess);
```

```
using var provider = services.BuildServiceProvider();
RecentGeneratorContextFactory.IGeneratorContextFactory factory = provider.
GetRequiredService<RecentGeneratorContextFactory.IGeneratorContextFactory>();
```

The process of injecting the ContextFactory.

Constructing the Context

An instance of the context is created using the necessary Ember objects.

```
LabOrder labOrder = new LabOrder("1", "Pediatrics");
Vaccine vaccine = new Vaccine("Polio", 1, DateTime.UtcNow);

RecentIGeneratorContext ctx = factory.CreateGeneratorContext(labOrder, vaccine);
```

Executing the script

A script can be executed either by first getting it and then calling the methods on the retrieved object or directly within one line as shown in the second example. The first method is the more desirable option, as it is cleaner and more readable.

```
ActionScript scriptInstance = _scriptManager.GetScript<ActionScript>(script.ScriptName!);
RecentActionResult ar = await scriptInstance.ExecuteAsync(ctx);
```

The desirable “slow” way where the retrieval and the execution are split into two method calls.

```
RecentActionResult ar = (RecentActionResult)await _scriptManager!.ExecuteScript<IActionScript>
("AddPediatricTestsV2", ctx, nameof(RecentIActionScript.ExecuteAsync));
```

The less desirable “quick” way where the retrieval and execution are handled in one line.

At first, the approach of first having to call a Get method to be able to execute a script might seem weird since one could unite the get and execute calls in one method call, but after reconsidering how the system is used, I realized that the action of having to retrieve (calling “Get”) a script is much rarer than calling a method on a script (executing a specific method), therefore it would introduce less complexity for the future developers as they only have to retrieve the script once in the beginning of a method for example, and then will be able to use this script façade object to simply call the script methods instead of repeatedly having to retrieve the script.

Within this test file I also added some showcase integration tests that mock scenarios where a method was renamed, an old method was deleted in a newer version of the script interface, and where a new method was added to the interface.

Validation and Evaluation

This chapter focuses on the validation and evaluation of the Scripting Proof of Concept, it analyzes the correctness of the system, its robustness, and most importantly the initial design constraints and if the requirements given to me were met.

Unit and Integration Testing

To ensure the system is functional and doesn't break in the future, it is essential to write tests for each piece of code that is added to the codebase. This process of continuously testing new code is called Test-Driven-Development (Fowler, Test Driven Development, 2023)).

The testing framework that was most convenient to use was MSTest. It was used to write primarily integration tests that cover the entire lifecycle of scripts written by customers, from creation and validation to compilation and execution. The testing strategy prioritizes integration tests, as they were superior for the use case that I was dealing with (small, lightweight PoC with a local DB running in a container). In retrospect, unit tests should have been used more as they would allow for simpler isolation and mocking of individual components, which would allow developers to test edge cases without the overhead of the database operations.

Script Lifecycle and Compilation Testing

Due to the extensive nature of the test suite (comprising approximately 2,500 lines of code), this chapter discusses only the most important tests. To ensure maintainability and reduce code duplication, a TestHelper component was created for commonly reused testing code.

Integration tests such as `CreateAndInsertCompiledCacheTest` and `UpdateAndCompileTest` validate that scripts including their compiled versions are correctly stored in the database. The validation logic written inside the `ScriptCompiler` is rigorously tested across multiple test classes such as for example in the [SecurityTest.cs](#) file, which ensures that invalid scripts correctly get rejected before being inserted into the database, it also ensures that the correct exceptions are thrown in the case of a script containing errors. Besides the validation logic inside the compiler, a script can also fail during the compilation step, in this case, the system throws an exception called "CompilationFailedException", which contains an `EmitResult` object, which can then be later used to extract the exact error data. This `EmitResult` object then allows the Ember developers to identify syntax or structural issues before the script is inserted in the database.

Execution and Versioning

One of the most important requirements of the system is maintaining compatibility across multiple API versions. The `EmberVersioningTests` and `ScriptVersionScannerTest` verify that the system correctly identifies and loads the correct script versions based on the metadata attributes that are defined above the corresponding interfaces. To ensure backward compatibility, the downgrade mechanism is tested inside the [CreateContextByDowngradeTest](#). This proves that if a script expects an older API version (e.g., `IGeneratorContextV1`), the system can successfully downgrade a newer context version (e.g., `IGeneratorContextV3`) to meet the script's expectations without throwing an exception during the execution stage. The core usage cases are validated through the `Crud.cs` file, where I defined the main usage scenarios such as Create, Get, Update, Delete, and Execute. This file not only serves as a test file for the main operations but also serves as documentation showing how the system should be used.

Performance Optimization

As already described in a previous chapter, the system avoids the significant overhead of Just-In-Time compilation on every execution by using an Ahead-Of-Time compilation strategy. Integration tests are then used to validate the caching mechanism ([GetCachesForEachApiVersionTestsAsync](#)), proving that the scripts in the form of byte arrays are retrieved from the database, proving that during standard execution the Roslyn compiler is not used.

Security and System Stability Rules

Because the customer scripts are always executed within the application domain of Ember, strict rules were enforced that prevent the system from accidentally executing or inserting scripts into the database that could lead to the system getting a performance hit or worse, even crashing the entire system.

1. **Timeouts and Infinite Loops:** The system ensures a script's execution time is strictly limited. This is necessary, for example, in the case of an infinite loop being executed within a script.
2. **Namespace and Type Restrictions:** The `ScriptCompiler` prevents unauthorized access to the system by banning namespaces. This logic is tested within the [SecurityTest.cs](#) file, which makes sure that scripts that try to use restricted namespaces or access forbidden types throw the correct `ForbiddenNamespaceException` or `ForbiddenTypeAccessException`.

3. **Data Isolation:** By forcing scripts to access internal classes through their respective interfaces (e.g., `ILabOrder`), the system guarantees that scripts can only access and modify these objects through the approved paths that are defined on those facades (the interfaces).

Limitations and Future Work

Limitations and Future Work

The final proof-of-concept successfully demonstrates that it is possible to build a secure, backwards compatible, and performant alternative to the previous Python-based scripting system using .NET 9 and C#. Nevertheless, it is important to note that in the current implementation there are still several limitations and things that should be addressed before the code is integrated into the actual Ember LIMS production codebase.

Security and Execution Constraints

One of the biggest points that should be addressed in the future is how the system handles scripts that contain loops, as I already described in a previous chapter the ScriptCompiler checks before each insertion into the database that the script checks within every loop if the cancellation token has been cancelled. This process and validation is absolutely crucial because without it, it could lead to scripts running forever without ever being aborted. The issue with this check is that it is not done in a really sophisticated way (it simply checks using the expression as a string), and it is absolutely possible for a customer to circumvent this check by using, for example, a goto statement. In the future it therefore would be beneficial to implement a smarter way to analyze the script's source code to absolutely guarantee that the token is checked. One way to do this would be implementing Control Flow Graph analysis using the .NET namespace FlowAnalysis, which would allow the ScriptCompiler to trace the actual execution paths of the code (Microsoft.CodeAnalysis.FlowAnalysis Namespace, 2026).

Another thing that should be addressed in the future is how namespaces and type usage is restricted, currently the system implements a short blacklist using a simple list of strings that successfully bans the most dangerous tools by analyzing the code. A safer approach would be switching out the blacklist for a whitelist that only allows scripts to use a strict set of explicitly approved tools.

In the future, one could explore completely isolating the scripts in a safe sandbox. This would be ideal because it guarantees safe script termination and strict data isolation. However, it would require significant effort, since the scripts need to access classes and objects within the Ember application. All this data would have to be constantly passed back and forth between the main application and the sandbox, which could severely decrease the performance of the system.

Caching and Performance Scalability

The first big part of optimizing the system is compiling the scripts once ahead-of-time instead of just-in-time before each execution, these pre-compiled scripts are then stored and queried from a PostgreSQL database. This works reasonably well for standard operations and probably could be integrated into the final Ember product, but nevertheless, it would be worth taking a hard look at a more optimized caching approach in the future to further optimize scenarios where the same script is executed many times in a row (like in a loop, for example).

Internal CI/CD and Testing Infrastructure

Currently the scripting system relies heavily on developers following rules defined by the scripting API. Some of those rules, such as every script interface having the correct attribute defined on it or no duplicate API versions inside the attributes, cannot be enforced at compile time and could throw errors at runtime. To avoid those errors only being discovered in the integration tests or worse by the customer during runtime, it would be beneficial to create custom Roslyn analyzers that run automatically before each Ember build to automatically detect those errors and mark them as warnings (Makarenko, 2025).

Furthermore, as the system was a proof of concept and completely separate from the Ember project the majority of tests that were written are integration tests that utilize a dockerized PostgreSQL database, in the future it would therefore be a good idea to prioritize separating the testing from the database and focus more on fast and isolated unit tests using mocking frameworks like Moq for example that ensure that internal components such as the ScriptCompiler can be tested without having to initialize a complete database context (Moule, 2023).

Developer Experience and Tooling

Since the proof of concept focuses only on the backend compilation, versioning, and execution. It therefore is still lacking a front-end user interface that would allow customers to write their custom scripts. This user interface can and should be built using the Blazor framework, which is a robust part of the .NET ecosystem. Using Blazor for the front-end would allow the usage of advanced code-editors such as, for example, the Monaco Editor which is the open-source base for VS Code. The integrated code-editor should then communicate directly with the ScriptCompiler of the backend to provide real-time syntax highlighting, compilation errors, and maybe even code suggestions while the customer is writing his script.

Furthermore, it would be very useful for the customer to set up some form of a testing environment where the customer can test out his script without risking to break something, specifically, this so-called “dry run” environment should let users execute their script against mocked data to verify the expected behavior and check for unforeseen logical errors before the script is ever inserted into the production database.

Memory Management

While the current system successfully controls the execution time of a script, there is still an issue with the memory management of the assembly of the script. The ScriptExecutor currently uses `Assembly.Load(byte[])` to execute scripts, which permanently loads the compiled script’s assembly in the Ember application domain. The .NET Garbage Collector cannot unload assemblies from this default domain (Ember application domain), even after a script finishes executing (Passant, 2014).

Because `Assembly.Load()` is called every time a script is executed, executing the same script repeatedly will load redundant, permanent copies of that assembly into the RAM of Ember. This is a small but nevertheless considerable waste of memory and leads to a memory leak. Because these assemblies can never be unloaded, the memory usage of the server running Ember will slowly increase until it requires a restart. Therefore, the final implementation that will be merged into the product should be refactored. One option would be refactoring the ScriptExecutor, so it caches the loaded assemblies and uses a `Collectible AssemblyLoadContext`. This allows for an isolated memory space, which allows the Garbage Collector to safely delete the assembly from the RAM of the server once the script was executed, which will prevent the system’s memory from being exhausted (`System.Runtime.Loader.AssemblyLoadContext`, 2026) (Jindal, 2025).

Conclusion

The main goal of this bachelor project was designing and implementing a modern, secure and backwards-compatible C# scripting system that replaces the previous Python implementation for the new Ember LIMS product. The proof-of-concept I created over the 12-week period uses .Net 9 and the Roslyn compiler platform and successfully provides an extensible and simple interface for future Ember developers. The proof-of-concept was achieved by strictly following programming principles such as Clean Architecture and Domain-Driven Design and design patterns such as the Façade, Factory, and Adapter pattern.

A significant challenge during the development process was balancing the power customer scripts have and system stability. To ensure scripts execute fast and do not have too much power, I implemented an Ahead-Of-Time compilation strategy combined with a compiler that rigorously tests if a customer script could harm the system's stability. Furthermore, the complex requirement of maintaining backwards compatibility over the constantly evolving API versions was solved by creating a robust versioning system that relies on upgrade and downgrade methods that are defined by the developers.

While the PoC successfully proves a C# based scripting system is possible, there are still clear limitations and issues that need future work before the system reaches production readiness. As I already noted in the according chapter, resolving issues such as the slow memory leak in the ScriptExecutor or the introduction of custom Roslyn analyzers will improve the system substantially.

Ultimately, this project proves that securely executing C# scripts created by customers within the Ember application is not only possible but highly beneficial. The proof of concept will serve as a strongly typed, performant, and reliable base that successfully fulfills the initial project requirements and helps prepare Ember for its release to customers.

References

- Cancellation in Managed Threads*. (2026). Retrieved from learn.microsoft.com:
<https://learn.microsoft.com/en-us/dotnet/standard/threading/cancellation-in-managed-threads>
- compiler*. (2026, 5 5). Retrieved from pcmag.com:
<https://www.pcmag.com/encyclopedia/term/compiler>
- Conrad, G. (2026). *Scripting-Proof-of-Concept*. Retrieved from github.com:
<https://github.com/conrad-gilles/Scripting-Proof-of-Concept/blob/main/scripting/Execution/ScriptExecutor.cs>
- Conrad, G. (2026). *Scripting-Proof-of-Concept*. Retrieved from github.com:
<https://github.com/conrad-gilles/Scripting-Proof-of-Concept>
- ControlledExecution.Run(Action, CancellationToken) Method*. (2026). Retrieved from learn.microsoft.com:
<https://learn.microsoft.com/en-us/dotnet/api/system.runtime.controlledexecution.run?view=net-10.0>
- Development, testing, acceptance and production*. (2026). Retrieved from wikipedia.org:
https://en.wikipedia.org/wiki/Development,_testing,_acceptance_and_production
- Edgerson, R. (2025). *A Primer on Insecure Reflection Practices in Java and C# Applications*. Retrieved from srocketsecurity.com/: <https://www.srocketsecurity.com/blog/a-primer-on-insecure-reflection-practices-in-java-and-c-applications>
- Evans, C. (2009). *Professional Programming: DTAP – Part 1: What is DTAP?* Retrieved from phparch.com: <https://www.phparch.com/2009/07/professional-programming-dtap-%e2%80%93-part-1-what-is-dtap/>
- Evans, E. (2004). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Fowler, M. (2020). *Domain Driven Design*. Retrieved from martinfowler.com:
<https://martinfowler.com/bliki/DomainDrivenDesign.html>
- Fowler, M. (2023). *Test Driven Development*. Retrieved from martinfowler.com:
<https://martinfowler.com/bliki/TestDrivenDevelopment.html>

Fowler, M. (n.d.). *Repository*. Retrieved from <https://martinfowler.com/https://martinfowler.com/eaacatalog/repository.html>

Gamma, E. (1995). *Design patterns : elements of reusable object-oriented software*.

GeeksForGeeks. (2026, 4 7). *Factory method Design Pattern*. Retrieved from [geeksforgeeks.org: https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/](https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/)

Jindal, V. (2025, 1 19). *AssemblyLoadContext in C#*. Retrieved from [medium.com: https://medium.com/@vikkasjindal/assemblyloadcontext-in-c-bbaacd692989](https://medium.com/@vikkasjindal/assemblyloadcontext-in-c-bbaacd692989)

Macam, D. (2020). *Terminating Asynchronous Operations in .Net*. Retrieved from [medium.com: https://medium.com/@macam.daryl/your-guide-on-using-net-cancellation-and-cancellationtokensource-574ee755fa25](https://medium.com/@macam.daryl/your-guide-on-using-net-cancellation-and-cancellationtokensource-574ee755fa25)

Makarenko, D. (2025). *Writing your own roslyn analyzer: enhancing C# code analysis*. Retrieved from [medium.com: https://medium.com/@denmaklucky/writing-your-own-roslyn-analyzer-enhancing-c-code-analysis-f4b74696cfcd](https://medium.com/@denmaklucky/writing-your-own-roslyn-analyzer-enhancing-c-code-analysis-f4b74696cfcd)

Martin, R. C. (2012). *The Clean Code Blog*. Retrieved from [blog.cleancoder.com: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html](https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html)

Microsoft.CodeAnalysis.FlowAnalysis Namespace. (2026). Retrieved from [learn.microsoft.com: https://learn.microsoft.com/en-us/dotnet/api/microsoft.codeanalysis.flowanalysis?view=roslyn-dotnet-4.6.0](https://learn.microsoft.com/en-us/dotnet/api/microsoft.codeanalysis.flowanalysis?view=roslyn-dotnet-4.6.0)

Moule, E. (2023). *Using Moq: A Simple Guide to Mocking for .NET*. Retrieved from [codemag.com: https://www.codemag.com/Article/2305041/Using-Moq-A-Simple-Guide-to-Mocking-for-.NET](https://www.codemag.com/Article/2305041/Using-Moq-A-Simple-Guide-to-Mocking-for-.NET)

Passant, H. (2014). *Dynamic assembly loading, memory issue*. Retrieved from [stackoverflow.com: https://stackoverflow.com/questions/22658181/dynamic-assembly-loading-memory-issue](https://stackoverflow.com/questions/22658181/dynamic-assembly-loading-memory-issue)

Rudraksh, N. (2023). *medium.com*. Retrieved from <https://medium.com/@rudrakshnavaty/clean-architecture-7c1b3b4cb181>

Security Considerations for Reflection. (2026, 2 3). Retrieved from [learn.microsoft.com/: https://learn.microsoft.com/en-us/dotnet/framework/reflection-and-codedom/security-considerations-for-reflection](https://learn.microsoft.com/en-us/dotnet/framework/reflection-and-codedom/security-considerations-for-reflection)

Security rules. (2026, 2 3). Retrieved from [learn.microsoft.com/: https://learn.microsoft.com/en-us/dotnet/fundamentals/code-analysis/quality-rules/security-warnings](https://learn.microsoft.com/en-us/dotnet/fundamentals/code-analysis/quality-rules/security-warnings)

- System.Runtime.Loader.AssemblyLoadContext, A. (2026). Retrieved from learn.microsoft.com: <https://learn.microsoft.com/en-us/dotnet/core/dependency-loading/understanding-assemblyloadcontext>
- Task cancellation*. (2026). Retrieved from learn.microsoft.com: <https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-cancellation>
- Teru, S. (2023). *AoT vs JIT compilation*. Retrieved from medium.com: <https://medium.com/@sivakishore.teru/aot-vs-jit-compilation-c99e29cc733d>
- The .NET Compiler Platform SDK*. (2026, 5 6). Retrieved from learn.microsoft.com: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/>
- Ulelu, I. (2024). *Benefits & Drawbacks of Dependency Injection*. Retrieved from jetbrains.com: https://www.jetbrains.com/guide/go/tutorials/dependency_injection_part_one/benefits/
- w3sDesign. (2018, 1 12). *The GoF Design Patterns Reference*. Retrieved from <https://w3sdesign.com>: https://w3sdesign.com/GoF_Design_Patterns_Reference0100.pdf
- Woodward, J. (2016). *In-memory C# compilation (and .dll generation) using Roslyn*. Retrieved from josephwoodward.co.uk: <https://josephwoodward.co.uk/2016/12/in-memory-c-sharp-compilation-using-roslyn>